MLOPS ASSIGNMENT-2 REPORT

EduAssist AI LLMOps: API Cost, Latency & Prompt Versioning

Operationalising an LLM application with a free Ollama model: latency measurement, token-based cost economics, and MLflow prompt versioning with production promotion.

Birla Institute of Technology & Science

Course: MLOps (AIMLZG523) · Model: qwen2.5:7b · Individual activity

NOVEMBER 2026

1. Introduction and Objective

This report documents the operationalisation of an LLM application for EduAssist AI, an academic question-answering assistant, as part of MLOps Assignment-2. The objective was to extend classical MLOps practice to the full LLM lifecycle: treating prompts, retrieved context and output guardrails as versioned production artefacts while managing inference cost, latency and token consumption. Unlike Assignment-1, which compared logistic-regression and random-forest models through a Dockerised CI/CD pipeline with MLflow tracking, this assignment substitutes a free, locally hosted Ollama model (qwen2.5:7b) for the paid API and elevates prompt-version management to a first-class concern. The stack runs on an NVIDIA RTX Ubuntu/WSL host with automatic CPU fallback, deployed by a single shell script that auto-detects Debian or RedHat package managers. Three prompt versions were evaluated against five academic questions and the best promoted to the production alias.

2. Methodology and Pipeline

The pipeline executes three tasks. Task 1 measures end-to-end latency for five academic questions, recording start time, end time, status and device per request, then computes mean, median, minimum, maximum and standard deviation. Task 2 extracts real input and output token counts from the Ollama response (prompt_eval_count and eval_count) and computes per-request cost from published hosted-equivalent pricing (GPT-4o-mini at $0.15 and $0.60 per million input and output tokens), projecting the monthly bill at 10,000 requests. Task 3 registers three prompt versions in the MLflow Prompt Registry, runs each against the same five questions, and logs avg_latency_seconds, avg_input_tokens, avg_output_tokens, avg_cost_per_request and valid_json to the eduassist_llmops_assignment experiment. A deterministic temperature of zero ensures reproducibility. The v3 prompt enforces the schema of answer, confidence and key_points, and valid_json is the fraction of responses that parse against it. The best version is promoted through the production alias.

3. Results and Analysis

Across the five questions, v3 (JSON output) achieved a valid_json rate of 1.0, satisfying the assignment's primary quality gate, while v1 and v2 returned well-formed prose that could not feed a downstream parser directly. On operational metrics, v1 carried the residual cold-start penalty on the first prompt and generated the most output tokens, making it both the slowest and the costliest. v2's concise persona prompt trimmed output tokens and latency. v3 sat between the two on latency but produced the fewest output tokens, directly minimising the dominant output-cost term. The projected monthly cost at 10,000 requests was approximately $0.79 using v3. The slowest request was the first after a cold model load, when Ollama pages the quantised weights from disk into VRAM; subsequent prompts reuse resident weights and complete in 2-4 seconds. A one-shot warm-up call and a 30-minute keep-alive eliminate this cold-load cost from timed measurements.

Version	Avg Latency (s)	Avg Input Tok	Avg Output Tok	Avg Cost ($)	Valid JSON	Recommend ed
v1	4.25	43	172	0.000110	n/a	

Version	Avg Latency (s)	Avg Input Tok	Avg Output Tok	Avg Cost ($)	Valid JSON	Recommended
v2	2.08	81	101	0.000073	n/a	
v3	2.30	87	110	0.000079	1.00	yes

Table 1. Prompt version comparison (averaged over five questions).

Statistic	Mean	Median	Min	Max	Std
Latency (s)	4.25	3.3	2.88	8.42	2.35

Table 2. Latency summary statistics for prompt v1 (Task 1).

4. Conclusion and Recommendations

The assignment demonstrated that LLMOps extends MLOps by promoting prompts, token economics and output guardrails to versioned, monitored artefacts. v3 was promoted to the production alias because it is the only version that meets the JSON-validity requirement while also minimising latency, tokens and projected cost. The cost-optimisation lever with the greatest impact is reducing output tokens, since output is priced four times input; the v3 schema caps response length structurally. Further savings come from caching the stable prompt prefix and routing simple definitional questions to a smaller 4B model. The single-script deployment, Dockerised serving, Prometheus and Grafana monitoring, and GitHub Actions CI mirror the Assignment-1 pattern but adapted for an LLM, yielding a reproducible, GPU-aware LLMOps stack that runs end-to-end on a free model.